

Exercise 1: Mocking and Stubbing (Mockito)

Aim:

To test a service that depends on an external API using Mockito by creating a mock object and stubbing its methods.

ExternalApi.java

```
public interface ExternalApi {  
    String getData();  
}
```

MyService.java

```
public class MyService {  
    private ExternalApi api;  
  
    public MyService(ExternalApi api) {  
        this.api = api;  
    }  
  
    public String fetchData() {  
        return api.getData();  
    }  
}
```

MyServiceTest.java

```
import static org.mockito.Mockito.*;  
import static org.junit.jupiter.api.Assertions.*;  
import org.junit.jupiter.api.Test;  
import org.mockito.Mockito;  
  
public class MyServiceTest {
```

@Test

```
public void testExternalApi(){  
    ExternalApi mockApi=Mockito.mock(ExternalApi.class);  
    when(mockApi.getData()).thenReturn("Mock Data");  
    MyService service=new MyService(mockApi);  
    String result=service.fetchData();  
    assertEquals("Mock Data",result);  
}  
}
```

Explanation

- Mockito.mock() creates a mock object.
 - when(...).thenReturn(...) stubs the method to return a predefined value.
 - assertEquals() verifies that the mocked value is returned.
-

Output

JUnit Test Passed

Expected : Mock Data

Actual : Mock Data

BUILD SUCCESSFUL

Result: The external API is successfully mocked, the getData() method is stubbed to return "Mock Data", and the test passes successfully.

Exercise 2: Verifying Interactions (Mockito)

Aim:

To verify that a method is called with specific arguments using Mockito.

ExternalApi.java

```
public interface ExternalApi{  
    String getData();  
}
```

MyService.java

```
public class MyService{  
    private ExternalApi api;  
  
    public MyService(ExternalApi api){  
        this.api=api;  
    }  
  
    public String fetchData(){  
        return api.getData();  
    }  
}
```

MyServiceTest.java

```
import static org.mockito.Mockito.*;  
import org.junit.jupiter.api.Test;  
import org.mockito.Mockito;  
  
public class MyServiceTest{  
    @Test  
    public void testVerifyInteraction(){
```

```
    ExternalApi mockApi=Mockito.mock(ExternalApi.class);  
    MyService service=new MyService(mockApi);  
    service.fetchData();  
    verify(mockApi).getData();  
}  
}
```

Explanation

- Mockito.mock() creates a mock object.
 - service.fetchData() invokes the getData() method.
 - verify(mockApi).getData() checks that getData() was called exactly once.
-

Output

JUnit Test Passed

Verified: getData() method invoked successfully.

BUILD SUCCESSFUL

Result: Mockito successfully verifies that the getData() method was invoked on the mocked ExternalApi object.